Pico-LTR559-Module

Firmware Version 1.01
User Guide
Updated May 13th, 2026

Add-on C-Language module to merge with to your existing Raspberry Pi Pico (C-Language) program / project to support a LTR559 sensor, using I2C communication protocol.

IMPORTANT :

This User Guide is about Pico-LTR559-Module Firmware Version 1.01 from Andre St-Louys.
To get the full potential of this module, read this User Guide and carefully follow the indicated setup procedures.

Join our Pico-LTR559-Module discussion group on:
https://github.com/astlouys/Pico-LTR559-Module/discussions

Pico-LTR559-Module User Guide

Index

Index 2
Introduction 3
Development environment 4
Pico-LTR559-Module setup 5
Setup part 1: Cloning / installing the Pico-LTR559-Module to your system. 5
Setup part 2: Integrating the Pico-LTR559-Module to one of your projects. 7
Pico-LTR-Example 9

Pico-LTR559-Module User Guide

Introduction

The Pico-LTR559-Module is a C-Language add-on module that will allow your existing Raspberry Pi Pico C-Language program or project to use a LTR559 sensor to read ambient light level. It is also a (very close) proximity sensor.

To help you figure out the details on how to use the Pico-LTR559-Module, an example is included in the repository: « Pico-LTR559-Example ». This is a simple C-Language program making use of the Pico-LTR559-Module. This program example is also described later in this User Guide.

To get the full potential of the Pico-LTR559-Module, make sure you carefully follow the instructions given in this User Guide.

1) The « Setup part 1 » covers the steps required to install the Pico-LTR559-Module and Pico-LTR559-Example on your system. Make sure everything is properly installed and that you can build and run the Pico-LTR559-Example program without problem. You can then go on with the next step below to merge the add-on module to your own program.

2) The « Setup part 2 » covers the steps to follow when you want to add the Pico-LTR559-Module to one of your own existing Raspberry Pi Pico C-Language program.

You can join other users of the Pico-LTR559-Module on the discussion group on:
https://github.com/astlouys/Pico-LTR559-Module/discussions
I can also be reached with my personal email address at:
astlouys@gmail.com

Development environment

The instructions given below assume that your development system is a Raspberry Pi running the Raspberry Pi O.S. (formerly called « Raspbian ») and that you're using Visual Studio Code. They also assume that you followed the recommended naming and structure conventions for the directory structure on your development system. If this is not the case, you will have to adapt the instructions accordingly. Finally (and obviously), it is also assumed that you are already familiar with basic concepts like: how to program in C-Language, how to build a C-Language program on your development environment and how to download the resulting program to a Pico… (and that kind of things...)

The setup instructions will guide you to create a « /home/pi/pico/UTILITIES » directory to keep files / functions / modules that are often shared between my different projects.

Pico-LTR559-Module setup

Setup part 1: Cloning / installing the Pico-LTR559-Module to your system.

1) Create the project directory on your system:
```
mkdir  /home/pi/pico/Pico-LTR559-Module
```

2) Clone / copy all files from the GitHub repository to the « /home/pi/pico/Pico-LTR559-Module » project directory created in step 1 above.

3) If it does not already exist, create a « UTILITIES » directory in the pico directory.
```
mkdir  /home/pi/pico/UTILITIES
```

4) Move the following files from the Pico-LTR559-Module project directory to the UTILITIES directory created above:
```
mv  /baseline.h  /home/pi/pico/UTILITIES/
mv  /get_pico_identifier.c  /home/pi/pico/UTILITIES/
mv  /input_string.c  /home/pi/pico/UTILITIES/
mv  /log_printf.c  /home/pi/pico/UTILITIES/
```

5) Create a symbolic link from the project directory to the following files:
```
ln  -s  /home/pi/pico/pico-sdk/external/pico_sdk_import.cmake
ln  -s  /home/pi/pico/UTILITIES/baseline.h
ln  -s  /home/pi/pico/UTILITIES/get_pico_identifier.c
ln  -s  /home/pi/pico/UTILITIES/input_string.c
ln  -s  /home/pi/pico/UTILITIES/log_printf.c
```

6) In the « /Pico-LTR559-Example.c » and « /Pico-LTR559-Module.h » files, make sure the following parameters correspond to the actual configuration of your own LTR559 sensor:
```
StructLTR559.I2CPort
StructLTR559.GpioSDA
StructLTR559.GpioSCL
StructLTR559.ReadAddress
StructLTR559.WriteAddress
```

   NOTE: We could have expected to see all those parameters defined in the Pico-LTR559-Module.h file. However, since we may want to use this generic module amongst different projects (with each project having a different configuration), we believe that it is better to keep some of those parameters inside the project file itself.

7) Make sure the LTR559 VCC pin is properly connected to the Pico's « 3V3 » pin # 36 (3.3 volts, not 5 volts !!) and the LTR559 ground pin is also be connect to one of the Pico's ground pins.

8) This completes the first part of the setup. At this point, make sure you are able to build and use the Pico-LTR559-Example Firmware. Refer to the section of this User Guide on how to use it.

Setup part 2: Integrating the Pico-LTR559-Module to one of your projects.

Make sure you carefully followed the instructions in the « Setup part 1 » above, since they are a prerequisite for this section.

1) From your project directory: « /home/pi/pico/Pico-My-Project », create a symbolic link pointing to the following files:

```
ln  -s  /home/pi/pico/UTILITIES/baseline.h
ln  -s  /home/pi/pico/UTILITIES/get_pico_identifier.c
ln  -s  /home/pi/pico/UTILITIES/input_string.c
ln  -s  /home/pi/pico/UTILITIES/log_printf.c
ln  -s  /home/pi/pico/Pico-LTR559-Module/Pico-LTR559-Module.c
ln  -s  /home/pi/pico/Pico-LTR559-Module/Pico-LTR559-Module.h
ln  -s  /home/pi/pico/pico-sdk/external/pico_sdk_import.cmake
```

2) In your project file: « /home/pi/pico/Pico-My-Project.c », add the following include file:

```
#include "baseline.h"
#include "Pico-LTR559-Module.h"
```

3) In your project file: « /home/pi/pico/Pico-My-Project.c », add the function prototype for the following functions (along with the other function prototypes of your own program):

```
get_pico_identifier();
input_string();
log_printf();
```

You may simply open each corresponding « .c » file and make a copy-paste of the first function line, while not forgetting to add a semi-colon at the end for the function prototype.

4) In your project file: « /home/pi/pico/Pico-My-Project.c », declare the structure defined in the Pico-LTR559-Module.h file as a global variable:

```
struct  struct_ltr559  StructLTR559;
```

5) In your project file: « /home/pi/pico/Pico-My-Project.c », add the following variable declarations, definitions and function call. Make sure you properly setup the parameters to match those of your own:

```
UCHAR PicoIdentifier[40];
UCHAR PicoUniqueId[25];
UINT8 PicoType;
StructLTR559.I2CPort = ????;  // I2C port of the Pico
StructLTR559.GpioSDA = ????;  // I2C data line
StructLTR559.GpioSCL = ????;  // I2C clock line
StructLTR559.ReadAddress  = 0x????;
StructLTR559.WriteAddress = 0x????;
StructLTR559.LightLevelGain = ????;
get_pico_identifier(PicoUniqueId, PicoIdentifier, &PicoType);
```

NOTE:
You must define the GPIOs used to communicate with the LTR559 sensor. The Pico-LTR559-Module uses the I2C communication protocol. It can be seen as a « drop address » kind of protocol. So, as long as the I2C devices have different addresses, they can coexist on the same communication line. The lines SDA (data line) and SCL (clock line) must be set for the Pico's GPIOs you want to use, as well as the I2C_PORT. GPIO ports are suggested at the beginning of the source code. Those GPIOs will be used throughout some of my projects in the future and it may make your life easier if you stick with those parameters.

6) In your program file « /home/pi/pico/Pico-My-Project.c », include the source code for the following functions. Since those files contain the function source code, they must be included in the main body of program file, as would normally appear the functions:

```
#include  "get_pico_identifier.c"
#include  "input_string.c"
#include  "log_printf.c"
```

7) Your CMakeLists.txt file must be properly configured, for example to include the Pico-LTR559-Module.c file. You may want to copy the CMakeLists.txt file from the Pico-LTR559-Module directory to your project directory and modify it accordingly instead of starting from scratch.

Pico-LTR-Example

The « Pico-LTR559-Example » application is very simple... Its main purpose is to show how to integrate the « Pico-LTR559-Module » to your current Raspberry Pi Pico program / project. Nonetheless, here is how to use it.

Build the Pico-LTR559-Example application and download it to your Pico. The application will automatically be launched when uploaded to the Pico.

Once started, the application gives you some time to start a terminal emulator program and make a connection with the Pico through the USB CDC port. Since the purpose of the application is to read and display the LTR559 ambient light level to a terminal emulator, it is not very useful if you do not connect such a terminal.

Once this is done, you will be asked to enter date and time so that the Pico's real-time clock may be properly initialized and set. From that point on, all log messages will be time-stamped. The LTR559 will then be initialized. At this point, the ambient light level and proximity detected value will be refreshed on the log screen every 15 seconds. You may also press « Enter » and you have the choice to trigger a read cycle sooner and / or to toggle the Pico in upload mode.

A few words about proximity sensor... Don't expect to be able to use the proximity sensor integrated in this sensor to build an alarm system. I used to write that this is a (very close) proximity sensor. For example, it is used in smart phone to detect that your phone is on your ear in order to disable the touch screen keyboard. So, it will detect your hand from a few cm only from the sensor. The result is simply a « raw » value (closer or further). Do not try to interpret it as centimeters or the like... In conclusion, once you build the Firmware to read the ambient light value, it is not much work to handle the proximity value as well, but personally I haven't seen a utility for it.

You may wonder why there is no mention of the BME280 in this User Guide since there is also a BME280 sensor on the Pimoroni « Multi-sensor »... This is simply because the Pico-BME280-Module is in another of my repositories and can be used in a way similar to the Pico-LTR559-Module.